Reinforcement Learning

Chapter 4: Function Approximation

Ali Bereyhi

ali.bereyhi@utoronto.ca

Department of Electrical and Computer Engineering

University of Toronto

Fall 2026

Back to Model-free RL: *Control*

+ *But, we have in general prediction and control problems. In which one are we going to use function approximation?*
– Well, we can use in both

Recall

We have two major problems in model-free RL

- *Prediction in which for a given policy π we evaluate values by sampling the environment*
- *Control in which after each interaction, we improve our policy aiming to converge to the optimal policy*

Recap: *Online Control ϵ-Greedy*

We have seen a typical control loop via ϵ-greedy algorithm

```
X_Control():
1: Initiate two random policies π and π̄
2: while π ≠ π̄ do
3:     q̂_π = X_QUpdate(π) and π ← π̄      via function approximation
4:     π̄ = ϵ-Greedy(q̂_π)
5: end while
```

We can realize this loop using function approximation

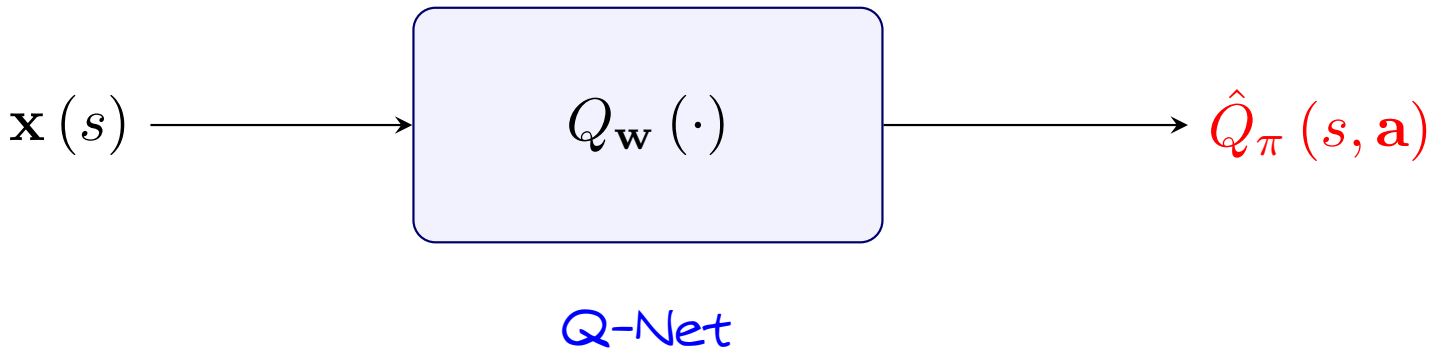
1. *Start with an initial approximator*
2. *Use the approximator to improve policy via ϵ-Greedy*
3. *Use SGD to update the approximation model from observation*

We need an action-value approximator in this case

Q-Net $\equiv$ *Action-Value Approximator – Form II*

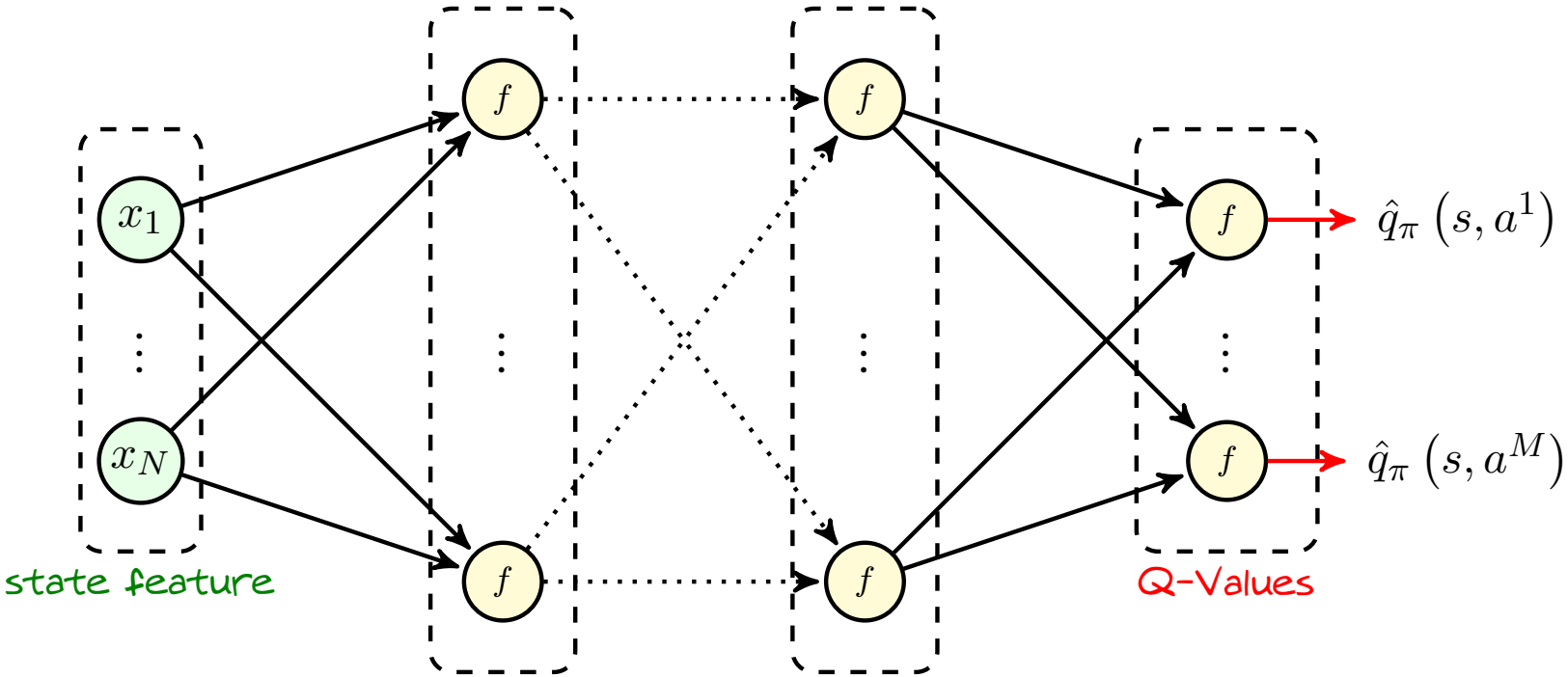
Q-Network

In the context of RL, the action-value approximation model that maps features to the vector of all action values is often referred to as Q-Net



Example: *MLP as DQN*

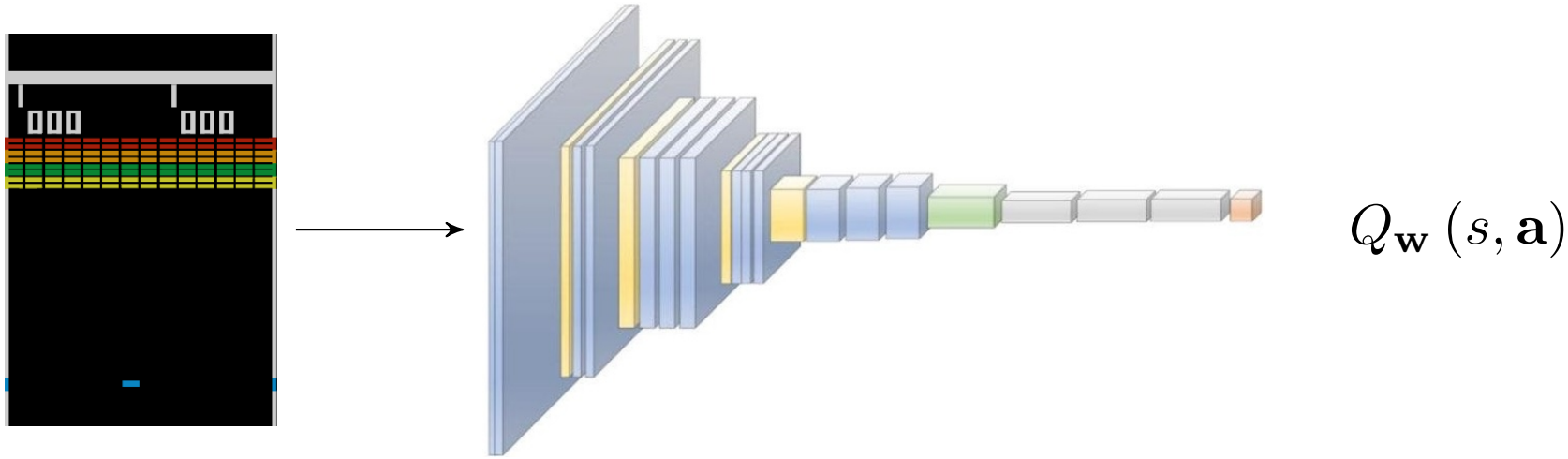
DQN can be simply an MLP



- *Give the state feature as an input vector*
- *The MLP estimates all action-values*

Example: *CNN as DQN*

It can be a convolutional neural network $\equiv$ *CNN*



- *Give the feature as an input tensor, e.g., when feature is a frame*
- *The CNN estimates all action-values*

Building Control Loop with Q-Net

We can use Q-Net to build a control loop: *similar to tabular RL, we can have on-policy or off-policy approaches*

- *Deep on-policy RL*
  - ↳ *We can use a DQN to realize an on-policy control loop, e.g., SARSA*
  - ↳ *They are less in use as compared to off-policy versions*
- *Deep off-policy RL*
  - ↳ *We may use a DQN to realize an off-policy control loop, e.g., Q-learning*
  - ↳ *Due to some reasons deep off-policy RL is more popular*
    - ↳ *We can use experience replay in this case*
    - ↳ *They are therefore more sample-efficient*
    - ↳ *No worries! We will see these reasons* ☺

Remark: *Value Network*

+ *But, when I look at internet, I usually see the term value network!*
– Sure! We have already worked with *value networks*

Value Network

Any approximation model that gets features and returns values is a value network; this value could be a state value or an action-value

- *Q-Net is a value network*
- $v_{\mathbf{w}}(\cdot)$ *that we used for prediction was also value network*

+ *Then, do we have any other sort of networks?!*
– Yes! We could have *policy networks*: *we will see them in the next chapter*

Recap: *Going On-Policy*

`SGD_TD_QEval(`λ`):`

Initialize with some initial $\mathbf{w}$ *and learning rate* α
for *episode* $k = 1 : K$ **do**
  Sample a trajectory $S_0, A_0 \xrightarrow{R_1} S_1, A_1 \xrightarrow{R_2} \cdots \xrightarrow{R_{T-1}} S_{T-1}, A_{T-1} \xrightarrow{R_T} S_T$
  for $t = 0 : T - 1$ **do**
    Compute $\Delta_t = R_{t+1} + \gamma v_{\mathbf{w}}(S_{t+1}) - Q_{\mathbf{w}}(S_t, A_t)$ `# forward propagation`
    Compute $\nabla_t = \nabla Q_{\mathbf{w}}(S_t, A_t)$ `# backpropagation`
    $E_{\mathbf{w}} \leftarrow \lambda\gamma E_{\mathbf{w}} + \nabla_t$
    Update weights as

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \Delta_t E_{\mathbf{w}}$$

  end for
end for

The key point in going on policy is to evaluate $v_{\mathbf{w}}(S_{t+1})$ *using our actual policy*

SARSA: *Going On-Policy*

We can modify line *line 5*: *we compute* $v_{\mathbf{w}}\left(S_{t+1}\right)$ *as*

$$v_{\mathbf{w}}\left(S_{t+1}\right) = \sum_{m=1}^{M} \pi\left(a^m | S_{t+1}\right) Q_{\mathbf{w}}\left(S_{t+1}, a^m\right)$$

In on-policy approach, we act before we update

our policy leads us to next action A_{t+1}

So, we could move on our policy and write

$$\pi\left(a|S_{t+1}\right) = \begin{cases} 1 & a = A_{t+1} \\ 0 & a \neq A_{t+1} \end{cases} \rightsquigarrow v_{\mathbf{w}}\left(S_{t+1}\right) = Q_{\mathbf{w}}\left(S_{t+1}, A_{t+1}\right)$$

SARSA with Q-*Net*

`SGD_SARSA()`:

Initialize with $\mathbf{w}$ *and learning rate* α
for *episode* $= 1 : K$ *or until* π *stops changing* **do**
  Initiate with a random state-action pair (S_0, A_0)
  for $t = 0 : T - 1$ *where* S_T *is either terminal or terminated* **do**
    Act A_t *and observe* $S_t, A_t \xrightarrow{R_{t+1}} S_{t+1}$
    Update policy to $\pi \leftarrow \epsilon$-`Greedy`$(Q_{\mathbf{w}}(S_t, A_t))$
    Draw the new action A_{t+1} *from* $\pi(\cdot|S_{t+1})$ *and move on policy*

$$S_t, A_t \xrightarrow{R_{t+1}} S_{t+1}, A_{t+1}$$

    Set $\Delta \leftarrow R_{t+1} + \gamma Q_{\mathbf{w}}(S_{t+1}, A_{t+1}) - Q_{\mathbf{w}}(S_t, A_t)$ `# forward propagation`
    Update $\mathbf{w} \leftarrow \mathbf{w} + \alpha \Delta \nabla Q_{\mathbf{w}}(S_t, A_t)$ `# backpropagation`
  end for
end for

SARSA with Q-*Net and Eligibility Tracing*

We have seen that *with function approximation eligibility tracing reduces to*

$$E_{\mathbf{w}} \leftarrow \gamma\lambda E_{\mathbf{w}} + \nabla Q_{\mathbf{w}}\left(S_t, A_t\right)$$

Let's fit it into our SARSA control loop!

Eligibility Tracing $\propto$ Backpropagation

If we use a DQN, we should backpropagate to compute the eligibility tracing: this point is intuitive as both approaches naturally follow the same logic

SARSA(λ) with *Function Approximation*

`SGD_SARSA(`λ`):`

Initialize with $\mathbf{w}$ *and learning rate* α
for *episode* $= 1 : K$ *or until* π *stops changing* **do**
  Initiate with a random state-action pair (S_0, A_0)
  for $t = 0 : T - 1$ *where* S_T *is either terminal or terminated* **do**
    Act A_t *and observe* $S_t, A_t \xrightarrow{R_{t+1}} S_{t+1}$
    Update policy to $\pi \leftarrow \epsilon$-`Greedy`$(Q_{\mathbf{w}}(S_t, \mathbf{a}))$
    Draw the new action A_{t+1} *from* $\pi(\cdot|S_{t+1})$ *and move on policy*

$$S_t, A_t \xrightarrow{R_{t+1}} S_{t+1}, A_{t+1}$$

    Set $\Delta \leftarrow R_{t+1} + \gamma Q_{\mathbf{w}}(S_{t+1}, A_{t+1}) - Q_{\mathbf{w}}(S_t, A_t)$ `# forward propagation`
    $E_{\mathbf{w}} \leftarrow \lambda\gamma E_{\mathbf{w}} + \nabla Q_{\mathbf{w}}(S_t, A_t)$ `# backpropagation`
    Update $\mathbf{w} \leftarrow \mathbf{w} + \alpha \Delta E_{\mathbf{w}}$
  end for
end for

Recap: *Going Off-Policy*

In off-policy control: *we behave with a policy π but update by a target policy $\bar{\pi}$*

- *We could use importance sampling to evaluate target policy*
- *We mainly focused on Q-learning approach*

Q-Learning

Q-learning is an off-policy TD control algorithm, where we sample with ϵ-greedy policy but update with greedy policy

Key property of Q-learning is that we don't really need importance sampling: we could directly update as

$$\hat{q}_{\bar{\pi}}\left(S_t, A_t\right) \leftarrow \hat{q}_{\bar{\pi}}\left(S_t, A_t\right) + \alpha\left(R_{t+1} + \gamma \max_m \hat{q}_{\bar{\pi}}\left(S_{t+1}, a^m\right) - \hat{q}_{\bar{\pi}}\left(S_t, A_t\right)\right)$$

Q-Learning with Q-*Net*

We can therefore extend out Q-learning algorithm to

`SGD_Q-Learning()`:

1: *Initialize with* $\mathbf{w}$ *and learning rate* α
2: ***for*** *episode* $= 1 : K$ *or until* π *stops changing* ***do***
3: *Initiate with a random state* S_0
4: ***for*** $t = 0 : T - 1$ *where* S_T *is either terminal or terminated* ***do***
5: *Update policy to* $\pi \leftarrow \epsilon$-`Greedy`$(Q_{\mathbf{w}}(S_t, \mathbf{a}))$
6: *Draw action* A_t *from* $\pi(\cdot|S_t)$ *and observe* $S_t, A_t \xrightarrow{R_{t+1}} S_{t+1}$
7: $\Delta \leftarrow R_{t+1} + \gamma \max_m Q_{\mathbf{w}}(S_{t+1}, a^m) - Q_{\mathbf{w}}(S_t, A_t)$ `# forward propagation`
8: *Update* $\mathbf{w} \leftarrow \mathbf{w} + \alpha \Delta \nabla Q_{\mathbf{w}}(S_t, A_t)$ `# backpropagation`
9: ***end for***
10: ***end for***

Incremental Algorithms: *Challenges*

What we have developed by now are the so-called *incremental approaches*

Incremental Algorithms

Incremental algorithms use the actual sample at each time to update the Q-Net

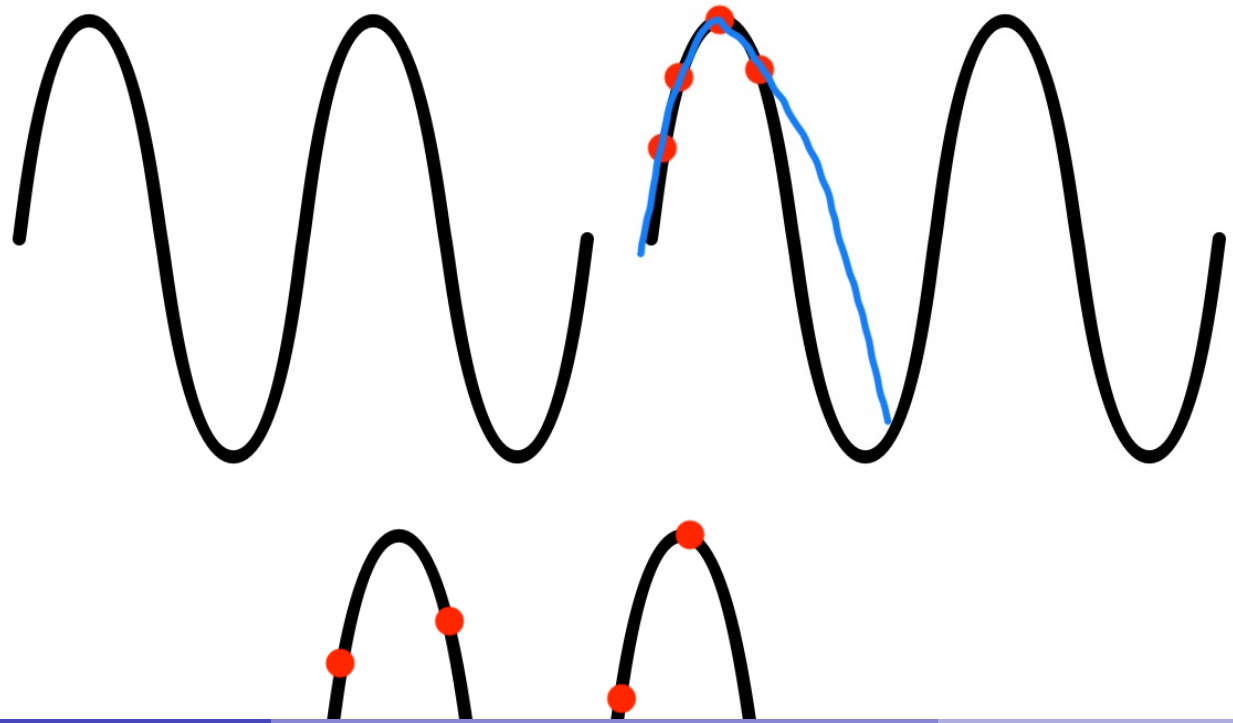
Though easy to implement, incremental approaches are not efficient

1. *They are extremely sample-inefficient*
   - ↳ *Once we use a sample, we are over with it*
   - ↳ *But, in deep learning we are used to use the same samples several times*
     - ↳ *We make a training loop with multiple epochs*
     - ↳ *In each epoch, we go through the whole dataset*

Incremental Algorithms: *Challenges*

Though easy to implement, incremental approaches are not efficient

2. *They use samples that are strongly correlated*
   - ↳ *In sample* $S_t, A_t \xrightarrow{R_{t+1}} S_{t+1}, A_{t+1}$ *states* S_t *and* S_{t+1} *are closely related*
     - ↳ *If state is the location; then, locations in time* t *and* $t+1$ *are very close*
   - ↳ *But, we know that we need independent samples*
     - ↳ *With correlated samples we can easily stick to a local fitting*
     - ↳ *Our Q-Net does not generalize very well*



Solution: *Batch Training*

The solution to these challenges is to use *batch training*

1. *Save every sample in a database*
   - ↳ *We can only save the values we need, i.e., estimator values*
2. *In each iteration sample from database*
   - ↳ *We can reuse (replay) our sampled experiences*
   - ↳ *We can reduce the correlation between succeeding samples*

This is what we call experience replay

- *This needs us to control off-policy*
  - ↳ *We are using samples of previous non-improved policies*
  - ↳ *We want to update the value of a different target policy*
- *This is why deep off-policy algorithms are more sample-efficient*

Vanilla Deep Q-Learning

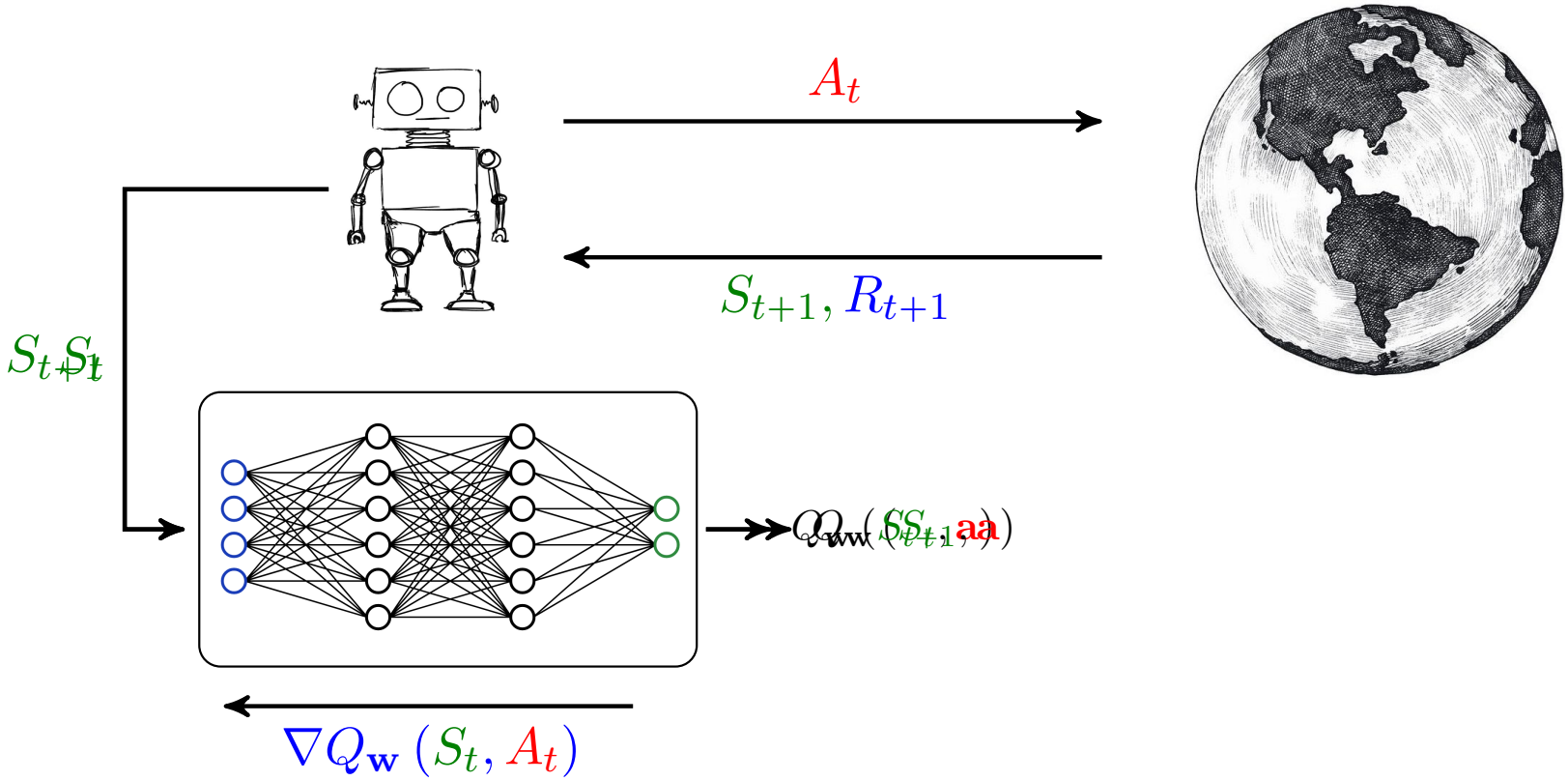
`SGD_Q-Learning()`:

1: *Initiate with* $\mathbf{w}$ *and learning rate* α
2: ***for*** *episode* $= 1 : K$ *or until* π *stops changing* ***do***
3: *Initiate with a random state* S_0
4: ***for*** $t = 0 : T - 1$ *where* S_T *is either terminal or terminated* ***do***
5: *Update policy to* $\pi \leftarrow \epsilon$-`Greedy`$(Q_{\mathbf{w}}(S_t, \mathbf{a}))$
6: *Draw action* A_t *from* $\pi(\cdot|S_t)$ *and observe* $S_t, A_t \xrightarrow{R_{t+1}} S_{t+1}$
7: $\Delta \leftarrow R_{t+1} + \gamma \max_m Q_{\mathbf{w}}(S_{t+1}, a^m) - Q_{\mathbf{w}}(S_t, A_t)$ `# forward propagation`
8: *Update* $\mathbf{w} \leftarrow \mathbf{w} + \alpha \Delta \nabla Q_{\mathbf{w}}(S_t, A_t)$ `# backpropagation`
9: ***end for***
10: ***end for***

In deep Q-learning, we use a DQN to perform offline control via Q-learning

$$\textit{deep Q-learning} \equiv \textit{DQL}$$

Vanilla DQL: *Visualization*



We update the weights on the DQN as $\mathbf{w} \leftarrow \mathbf{w} + \alpha \Delta \nabla Q_{\mathbf{w}}(S_t, A_t)$

$$\Delta \leftarrow \boxed{R_{t+1} + \gamma \max_m Q_{\mathbf{w}}(S_{t+1}, a^m)} - Q_{\mathbf{w}}(S_t, A_t)$$

Vanilla Deep Q-Learning: *Challenges*

Vanilla DQL does not perform *impressive*: *it suffers from two major challenges*

1. *We deal with strongly correlated samples*
   - ↳ *We handle this issue via experience reply*
2. *The labels in the sample data-points change in each iteration*

$$\Delta \leftarrow \boxed{R_{t+1} + \gamma \max_m Q_{\mathbf{w}}\left(S_{t+1}, a^m\right)} - Q_{\mathbf{w}}\left(S_t, A_t\right)$$

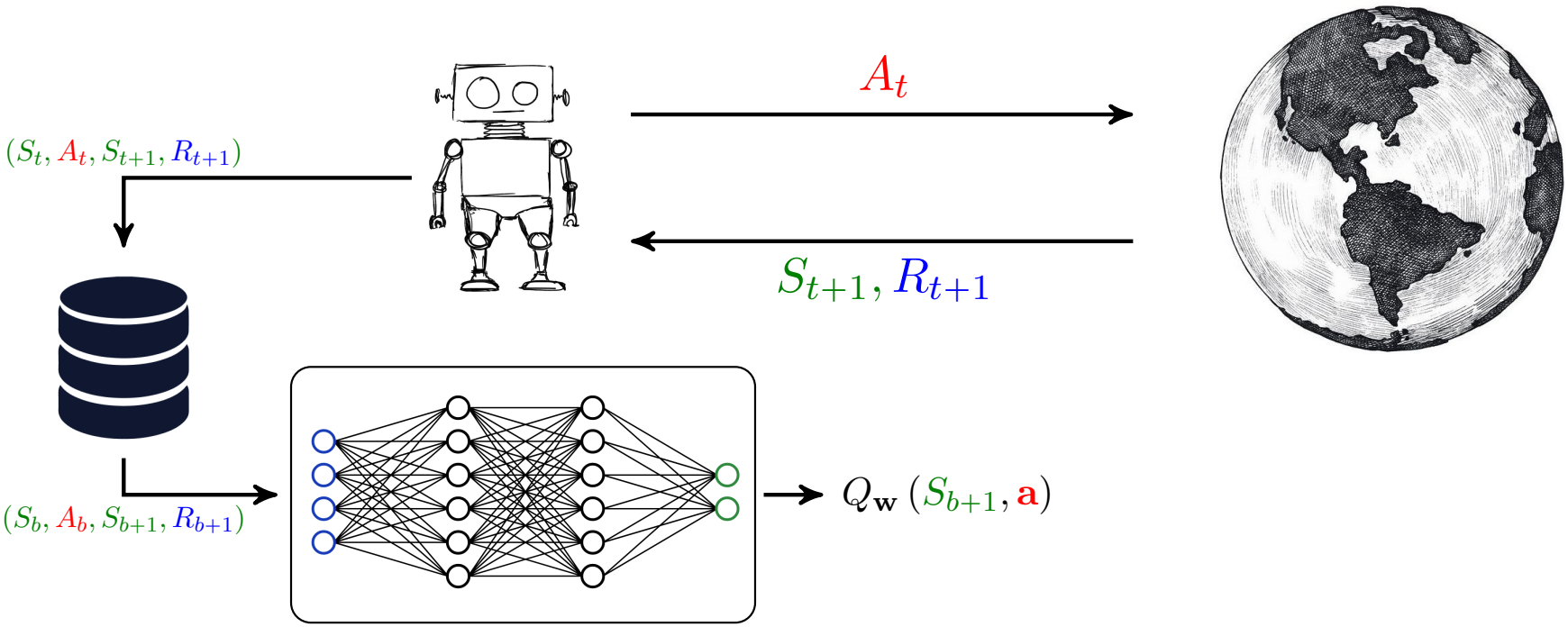
   - ↳ *Here, we can look at each sampled state as a data sample with label*

$$y_t = R_{t+1} + \gamma \max_m Q_{\mathbf{w}}\left(S_{t+1}, a^m\right)$$

   - ↳ *After each update of $\mathbf{w}$ this label changes*
   - ↳ *This results in divergence or high error variance*
   - ↳ *We are going to over-come this issue by using a target network*

Let's first get to experience replay

Experience Replay



We update DQN by mini-batches $\mathbf{w} \leftarrow \mathbf{w} + \sum_b \alpha \Delta_b \nabla Q_{\mathbf{w}}(S_b, A_b)$

$$\Delta_b \leftarrow \boxed{R_{b+1} + \gamma \max_m Q_{\mathbf{w}}(S_{b+1}, a^m)} - Q_{\mathbf{w}}(S_b, A_b)$$

DQL with Experience Replay

`DQL_v1()`:

1: *Initiate with* $\mathbf{w}$, *empty replay buffer* $\mathbb{D}$ *and learning rate* α
2: ***for*** *episode* $= 1 : K$ *or until* π *stops changing* ***do***
3: *Initiate with a random state* S_0
4: ***for*** $t = 0 : T - 1$ *where* S_T *is either terminal or terminated* ***do***
5: *Update policy to* $\pi \leftarrow \epsilon$-`Greedy`$(Q_{\mathbf{w}}(S_t, \mathbf{a}))$
6: *Draw action* A_t *from* $\pi(\cdot|S_t)$ *and observe* $S_t, A_t \xrightarrow{R_{t+1}} S_{t+1}$
7: *Add sample* $S_t, A_t \xrightarrow{R_{t+1}} S_{t+1}$ *to the replay buffer* $\mathbb{D}$
8: ***for*** *iteration* $\ell = 1 : L$ ***do***
9: *Sample mini-batch* $\mathbb{B} = \{S_b, A_b \xrightarrow{R_{b+1}} S_{b+1} \text{ for } b = 1 : B\}$ *from* $\mathbb{D}$
10: $\Delta_b \leftarrow R_{b+1} + \gamma \max_m Q_{\mathbf{w}}(S_{b+1}, a^m) - Q_{\mathbf{w}}(S_b, A_b)$
11: *Update* $\mathbf{w} \leftarrow \mathbf{w} + \alpha \sum_{b=1}^{B} \Delta_b \nabla Q_{\mathbf{w}}(S_b, A_b)$
12: ***end for***
13: ***end for***
14: ***end for***

DQL with Experience Replay

In general, we can iterate multiple mini-batches, i.e., $L > 1$

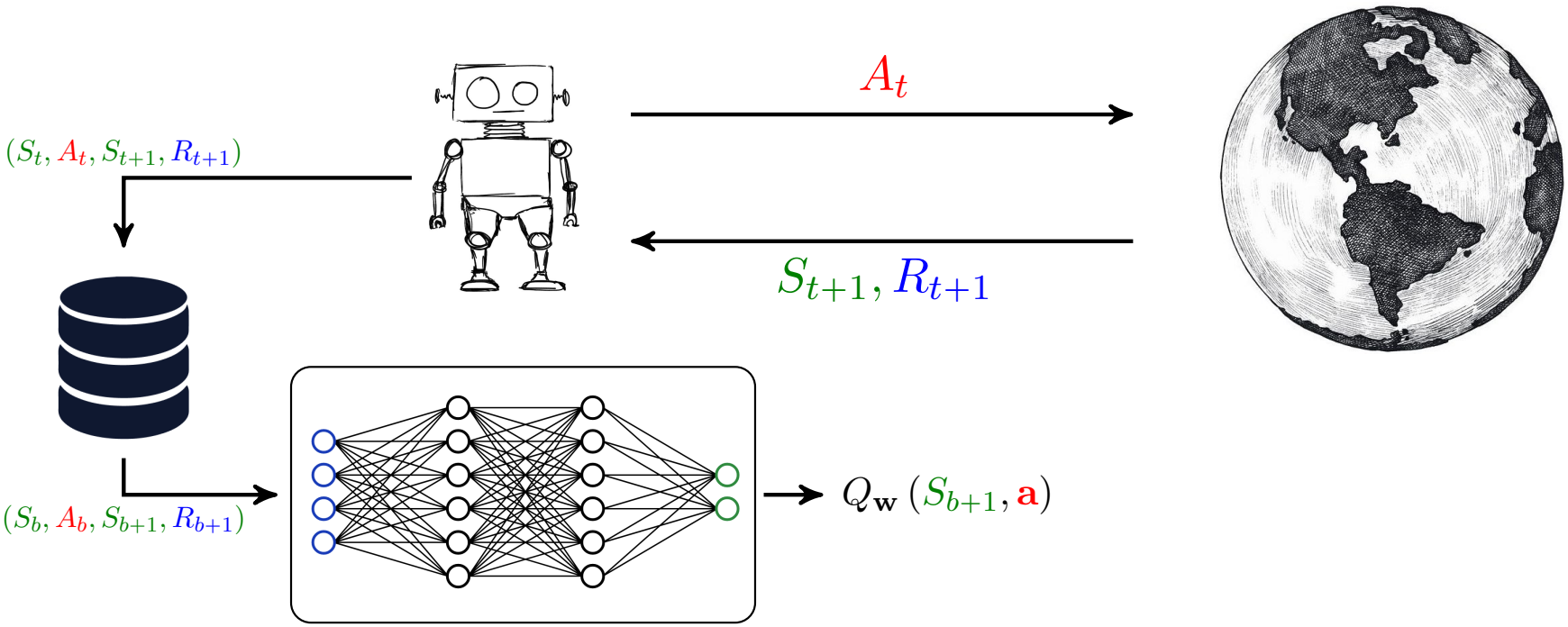
- *This can improve the convergence speed*
- *The trained DQN may however stick to a bad local minima*

In practice *we typically set* $L = 1$

- *with a relatively large batch-size we can see good convergence results*

+ *What about the second challenge? I didn't get really what was the issue at the first place!*
– Let's break it down!

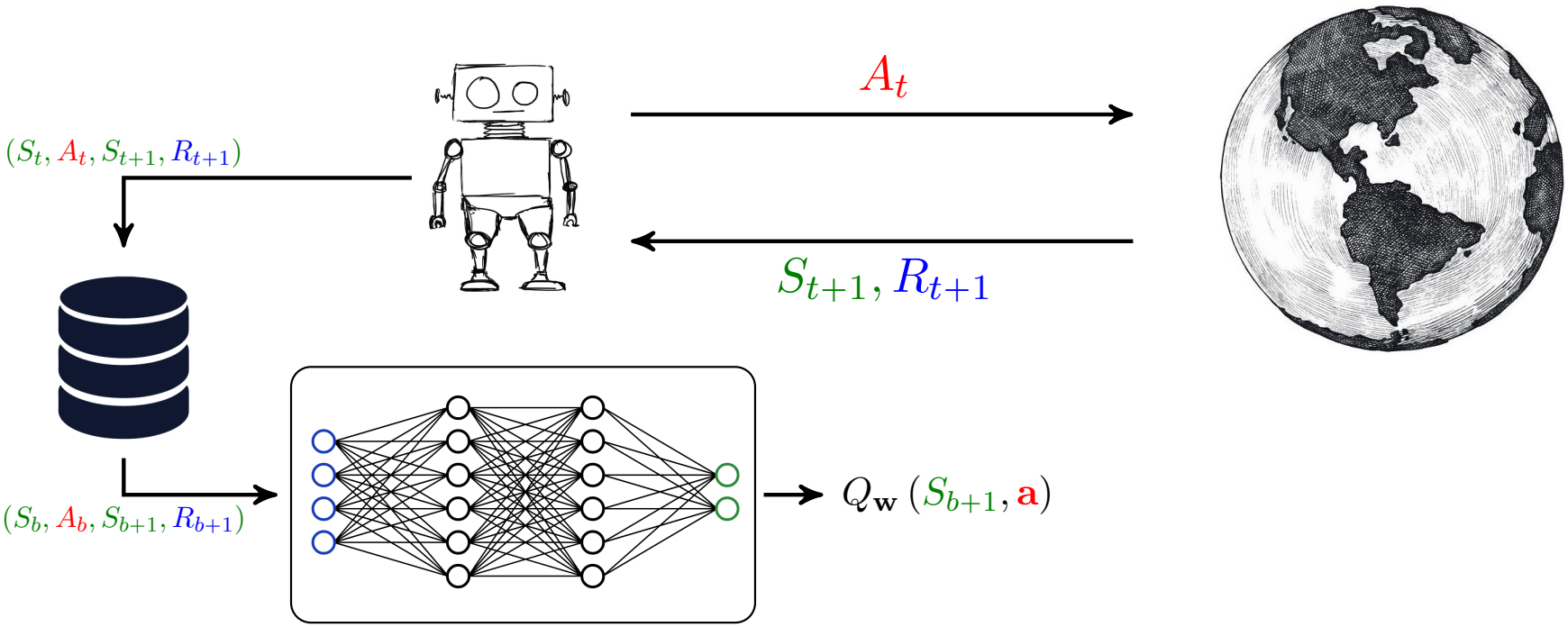
Varying Labels



We can look at the training procedure as supervised learning with label

$$y_b = R_{b+1} + \gamma \max_m Q_{\mathbf{w}}(S_{b+1}, a^m)$$

Varying Labels

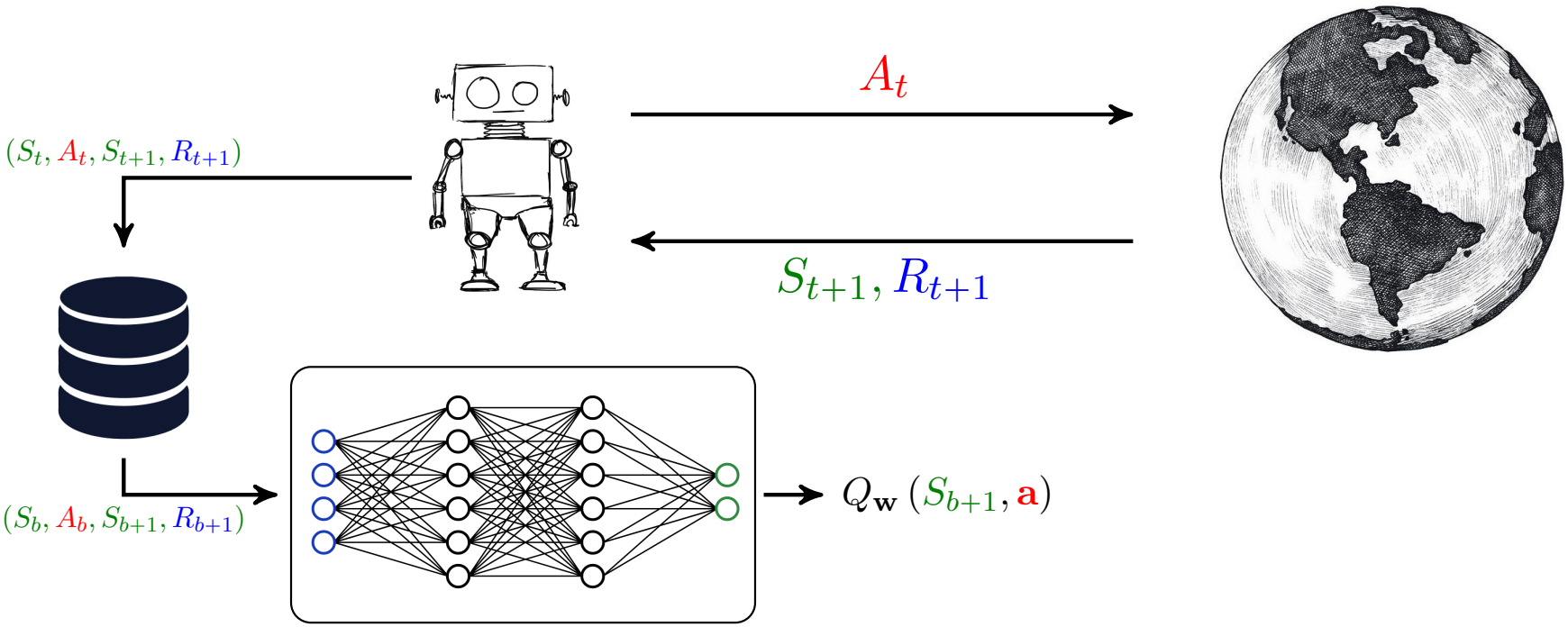


We are then updating $\mathbf{w}$ *gradually, such that*

$$\Delta_b = y_b - Q_{\mathbf{w}}(S_b, A_b)$$

shrinks: but, each time we update $\mathbf{w}$*, the label* y_b *also changes!*

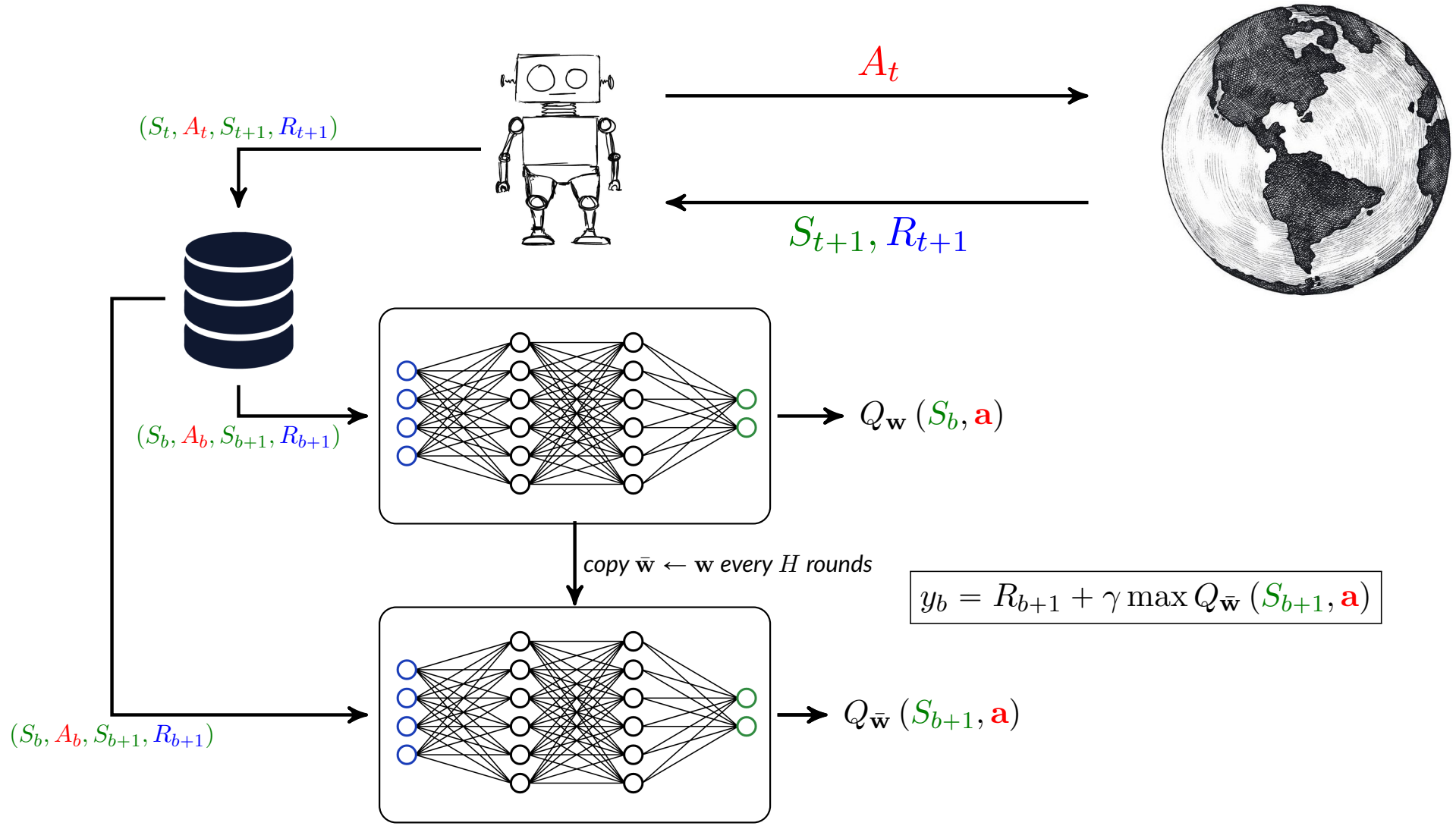
Varying Labels



This is an issue: in standard SGD, we have fixed labels and therefore

by multiple iterations the NN gradually converges to a good approximator

Target Network: *Simple Remedy*



DQL: *Classic Algorithm*

DQL():

Initiate with $\mathbf{w}$, *empty replay buffer* $\mathbb{D}$, *learning rate* α, *and a random state* S_t

while** interating **do

  Update $\bar{\mathbf{w}} \leftarrow \mathbf{w}$

  for $h = 1 : H$ ***do***

    $S_t \leftarrow S_{t+1}$ ***if*** S_t *is a terminal state* ***then*** *replace* S_t *with a random state*

    Update policy to $\pi \leftarrow \epsilon$-`Greedy`$(Q_{\mathbf{w}})$ *and draw* A_t *from* $\pi(\cdot|S_t)$

    Add $S_t, A_t \xrightarrow{R_{t+1}} S_{t+1}$ *to the replay buffer* $\mathbb{D}$

    for** iteration* $\ell = 1 : L$ ***do

      Sample mini-batch $\mathbb{B} = \{S_b, A_b \xrightarrow{R_{b+1}} S_{b+1} \text{ for } b = 1 : B\}$ *from* $\mathbb{D}$

      $\Delta_b \leftarrow \boxed{R_{b+1} + \gamma \max_m Q_{\bar{\mathbf{w}}}(S_{b+1}, a^m)} - Q_{\mathbf{w}}(S_b, A_b)$

      Update $\mathbf{w} \leftarrow \mathbf{w} + \alpha \sum_{b=1}^{B} \Delta_b \nabla Q_{\mathbf{w}}(S_b, A_b)$

    end for

  end for

end while

A Revolution: *Google DeepMind*

LETTER

doi:10.1038/nature14236

Human-level control through deep reinforcement learning

Volodymyr Mnih[1]*, Koray Kavukcuoglu[1]*, David Silver[1]*, Andrei A. Rusu[1], Joel Veness[1], Marc G. Bellemare[1], Alex Graves[1], Martin Riedmiller[1], Andreas K. Fidjeland[1], Georg Ostrovski[1], Stig Petersen[1], Charles Beattie[1], Amir Sadik[1], Ioannis Antonoglou[1], Helen King[1], Dharshan Kumaran[1], Daan Wierstra[1], Shane Legg[1] & Demis Hassabis[1]

Just to have a clue about how revolutionary it had been

Human-level control through deep reinforcement learning

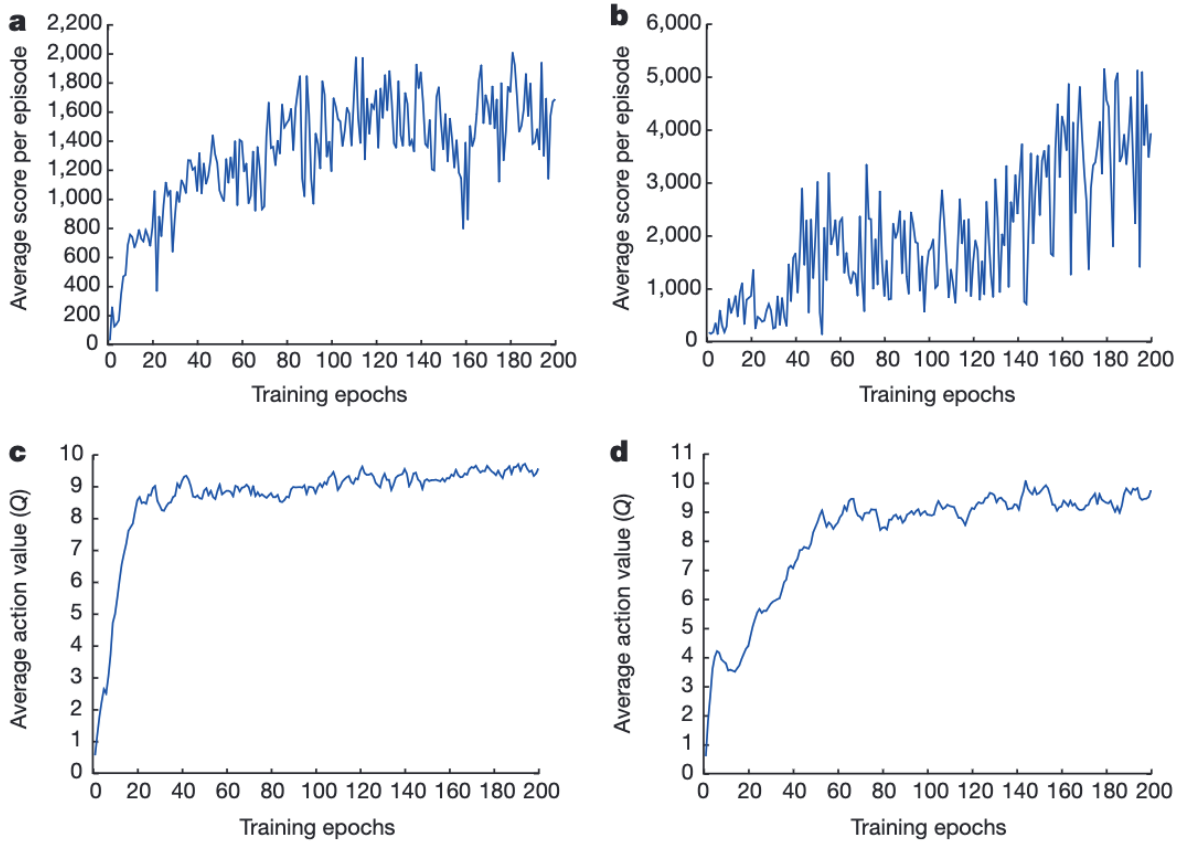
V Mnih, K Kavukcuoglu, D Silver, AA Rusu, J Veness… - nature, 2015 - nature.com

The theory of reinforcement learning provides a normative account 1, deeply rooted in psychological 2 and neuroscientific 3 perspectives on animal behaviour, of how agents may optimize their control of an environment. To use reinforcement learning successfully in situations approaching real-world complexity, however, agents are confronted with a difficult task: they must derive efficient representations of the environment from high-dimensional sensory inputs, and use these to generalize past experience to new situations. Remarkably …

☆ Save Cite Cited by 37969 Related articles All 57 versions Web of Science: 19263

A Revolution: *Google DeepMind*

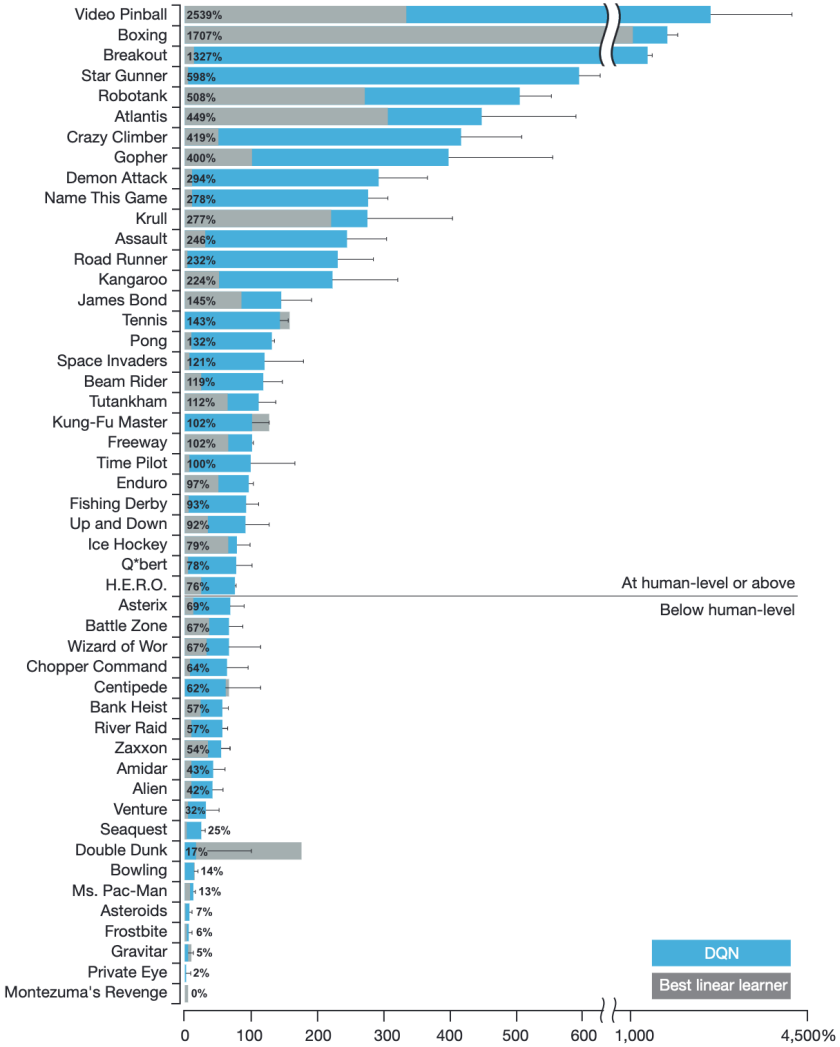
DQL could show extraordinary performance



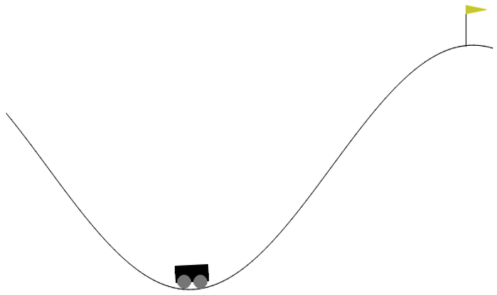
You may also watch the demo of the Breakout game on Youtube

A Revolution: *Google DeepMind*

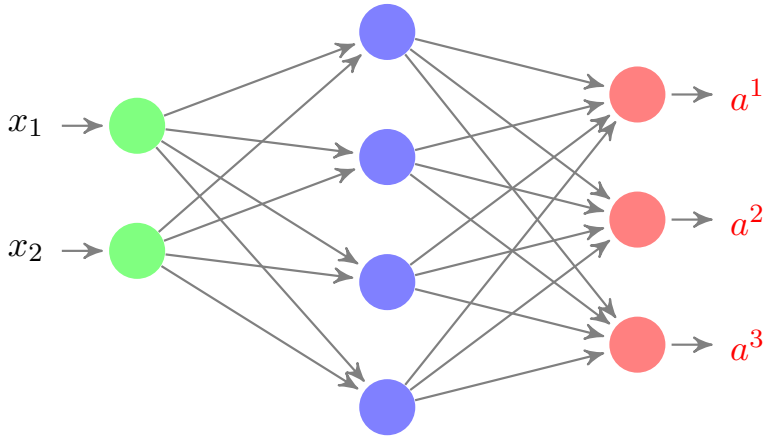
DQL was the first universal algorithm that could be applied to any environment



Example: *Mountain Car*



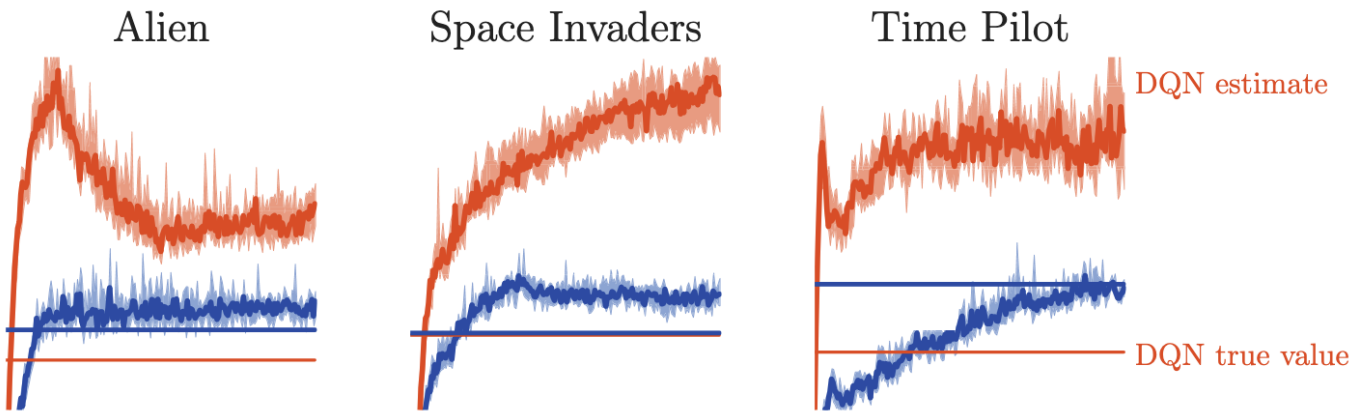
Let's try to imagine DQL in the mountain car example with a simple DQN



A more concrete example will be solved in the next Tutorial

DQL: *Bias Problem*

Q-learning is known to estimate action-values with bias: *take a look at few examples of DQL algorithm playing Atari games*



+ *Why is it happening? Is it simply because of TD approach?*
– It's severer than only TD: *it comes from the* $\max$ *operator in the update!*
+ *How does it come?*
– Well, It's best understood through an example

Bias Problem: *Basic Example*

Let's consider a simple example: *assume we are dealing with two computers, namely A and B. These computers are used to run the same program*

- *Computer A runs the program in exactly* $X_A = 8$ *seconds*
- *Computer B runs the program in exactly* $X_B = 5$ *seconds*

We however do not know these values: we can only run sample runs

- *Our time measurement is noisy, i.e., we compute on Computer* i

$$\hat{X}_i = X_i + \varepsilon$$

↳ *This error is random and can increase or decrease* X_i

Ultimate Goal

We want to compute the maximum runtime between the two computers

Bias Problem: *Basic Example*

We have access to noisy samples

$$\hat{X}_A^{(1)}, \ldots, \hat{X}_A^{(K)}, \hat{X}_B^{(1)}, \ldots, \hat{X}_B^{(K)}$$

If K is only moderately large, we could almost surely say that

$$\max\left\{\hat{X}_A^{(1)}, \ldots, \hat{X}_A^{(K)}, \hat{X}_B^{(1)}, \ldots, \hat{X}_B^{(K)}\right\} > \max\{X_A, X_B\} = 8$$

- *Within enough number of samples there is for sure a positive error sample*
- *This error sample renders an over-estimation*

A crucial point is that if we repeat this experiment several times, we always get an over-estimate; therefore,

> *after averaging over multiple instances, we are still biased!*

Solution to Max-Bias: *Double Measurements*

There is a very simple and intuitive solution to this problem: *we can collect two sequences of samples*

$$\textit{Sequence 1: } \hat{X}_{A,1}^{(1)}, \ldots, \hat{X}_{A,1}^{(K)}, \hat{X}_{B,1}^{(1)}, \ldots, \hat{X}_{B,1}^{(K)}$$

$$\textit{Sequence 2: } \hat{X}_{A,2}^{(1)}, \ldots, \hat{X}_{A,2}^{(K)}, \hat{X}_{B,2}^{(1)}, \ldots, \hat{X}_{B,2}^{(K)}$$

We find the index of the maximizer in Sequence 1, i.e.,

$$(i,k) = \operatorname{argmax} \left\{ \hat{X}_{A,1}^{(1)}, \ldots, \hat{X}_{A,1}^{(K)}, \hat{X}_{B,1}^{(1)}, \ldots, \hat{X}_{B,1}^{(K)} \right\}$$

But we take the sample from Sequence 2, i.e.,

$$\hat{X}_{\max} = \hat{X}_{i,2}^{(k)}$$

This is an unbiased estimator: if we repeat this experiment several times

> *after averaging over multiple instances, we get close to* X_A

DQL with *Double Measurements*

We can apply this idea to DQL: *we train two DQNs simultaneously*

- *We find out the action with maximum value using one DQN*
- *We evaluate the action-value of this action by the other DQN*

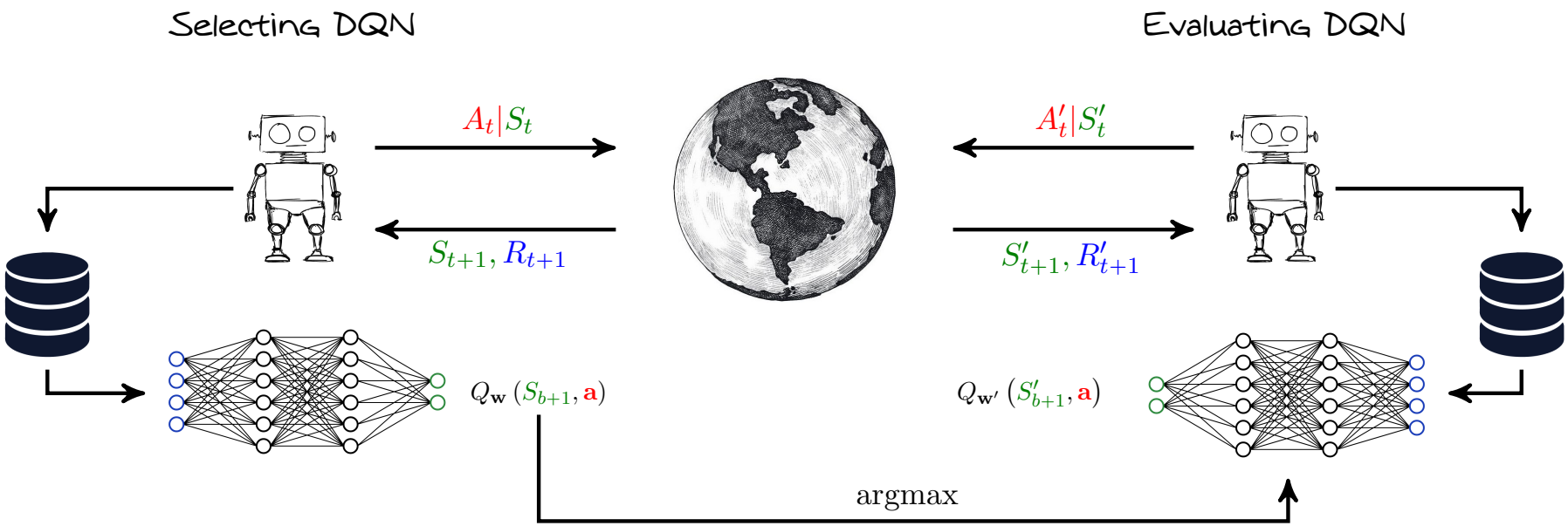
This way we get an unbiased estimator of the optimal action-value

we refer to this approach as double DQL

Attention

In general this does not mean that we are necessarily reaching to a better policy: we could have biased estimator and still play optimally!

Double DQL



We train two DQNs on two independent experience sets

- *With the online DQL we find*

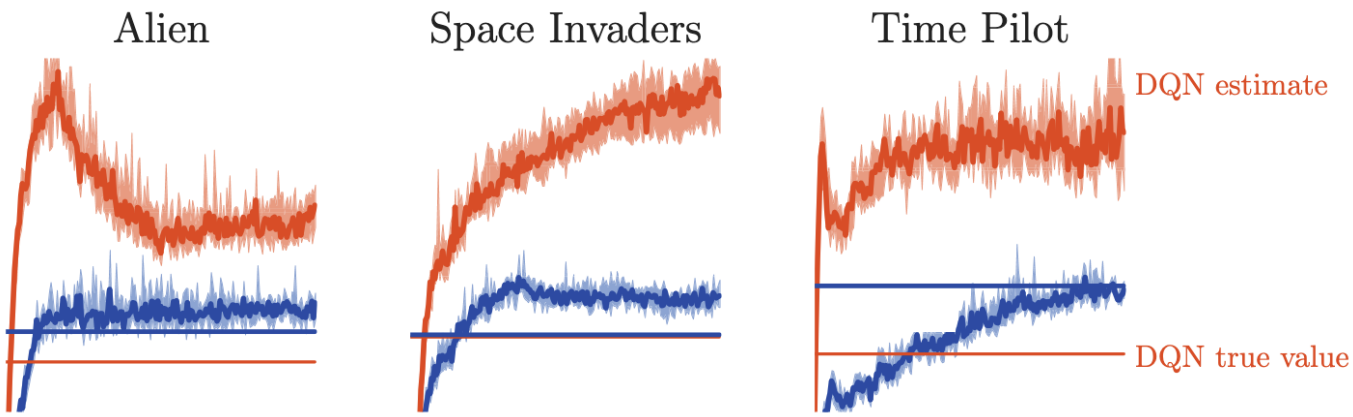
$$A^{\star}_{t+1} = \operatorname{argmax} Q_{\mathbf{w}}(S_{b+1}, \mathbf{a})$$

- *With the double DQL we evaluate*

$$y_b = R_{b+1} + \gamma Q_{\mathbf{w}'}\left(S_{b+1}, A^{\star}_{t+1}\right)$$

Double DQL: *Sample Results*

Let's take a look back to the earlier examples



Here, blue curves show double DQL

- *In all examples we are getting less bias as compared to DQL*
- *In two example it helps converging to better policy*
  - ↳ *Alien and Time Pilot*
- *In one example, it reduces bias but does not impact converging policy*

Other Variants

There are various extensions to classic DQL: *some famous ones are*

- *Double DQL*
  - ↳ *It tries to reduce the bias of value estimation*
- *Dueling DQL*
  - ↳ *It uses notion of advantage ≡ difference between action-value and value*
  - ↳ *It helps finding non-valuable actions*
- *Prioritized DQL*
  - ↳ *It gives priority to samples in the experience buffer*
- *Distributed DQL*
  - ↳ *It enables training DQN through pipelining*
  - ↳ *This let training of DQNs for massive problems*

Distributed DQL: *Gorila*

General Reinforcement Learning Architecture $\equiv$ *Gorila*

Gorila is implemented through four main generalization

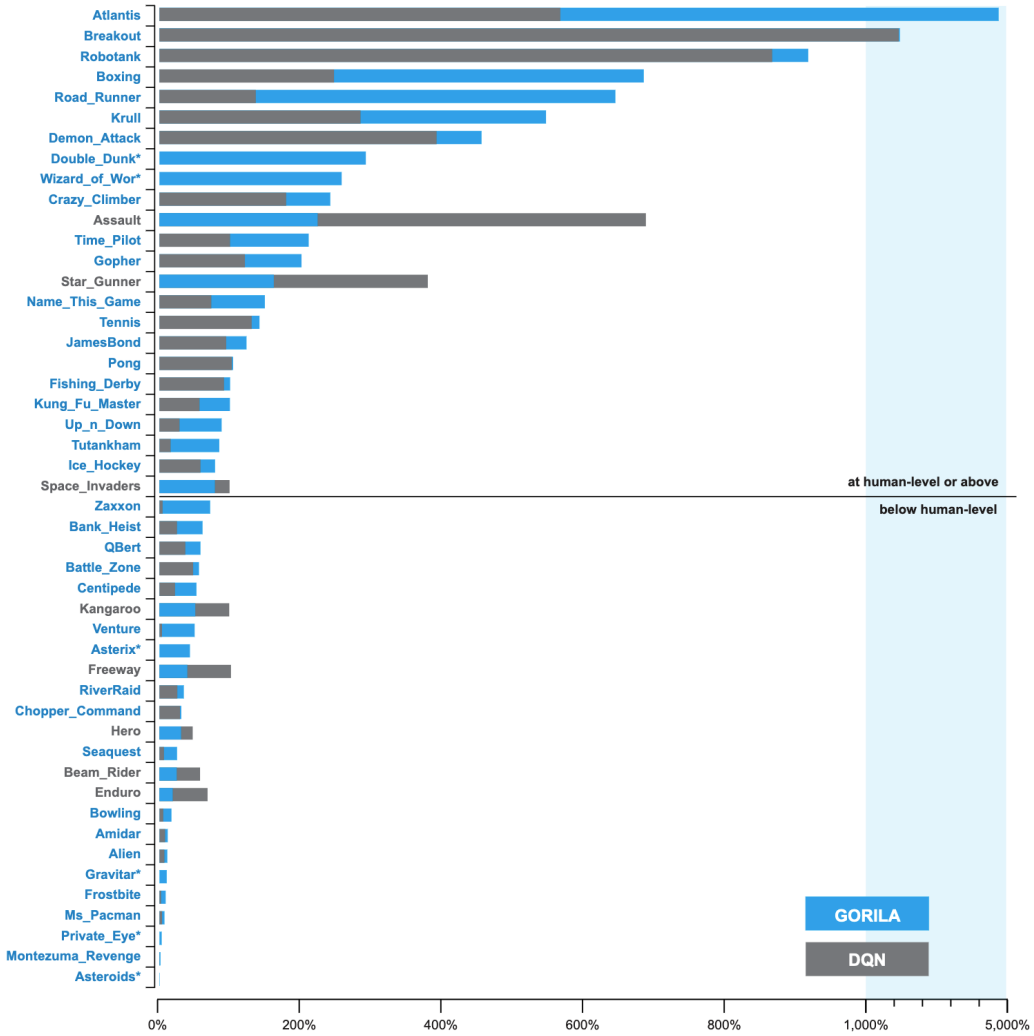
1. *Parallel actors generating acting behavior*
2. *Parallel DQNs trained by stored experience*
3. *Distributed storage of experience*
4. *A distributed DQN that specifies acting behavior policy*

Gorila massively parallelize implementation of DQL

this enables implementation of DQL for realistic hard control loops

Distributed DQL: *Gorila*

With significantly lower training time, Gorila starts to beat classic DQL



Dealing with Continuous Actions

Once DQL was established a new question raised

? *How can we handle settings with continuous action space?*

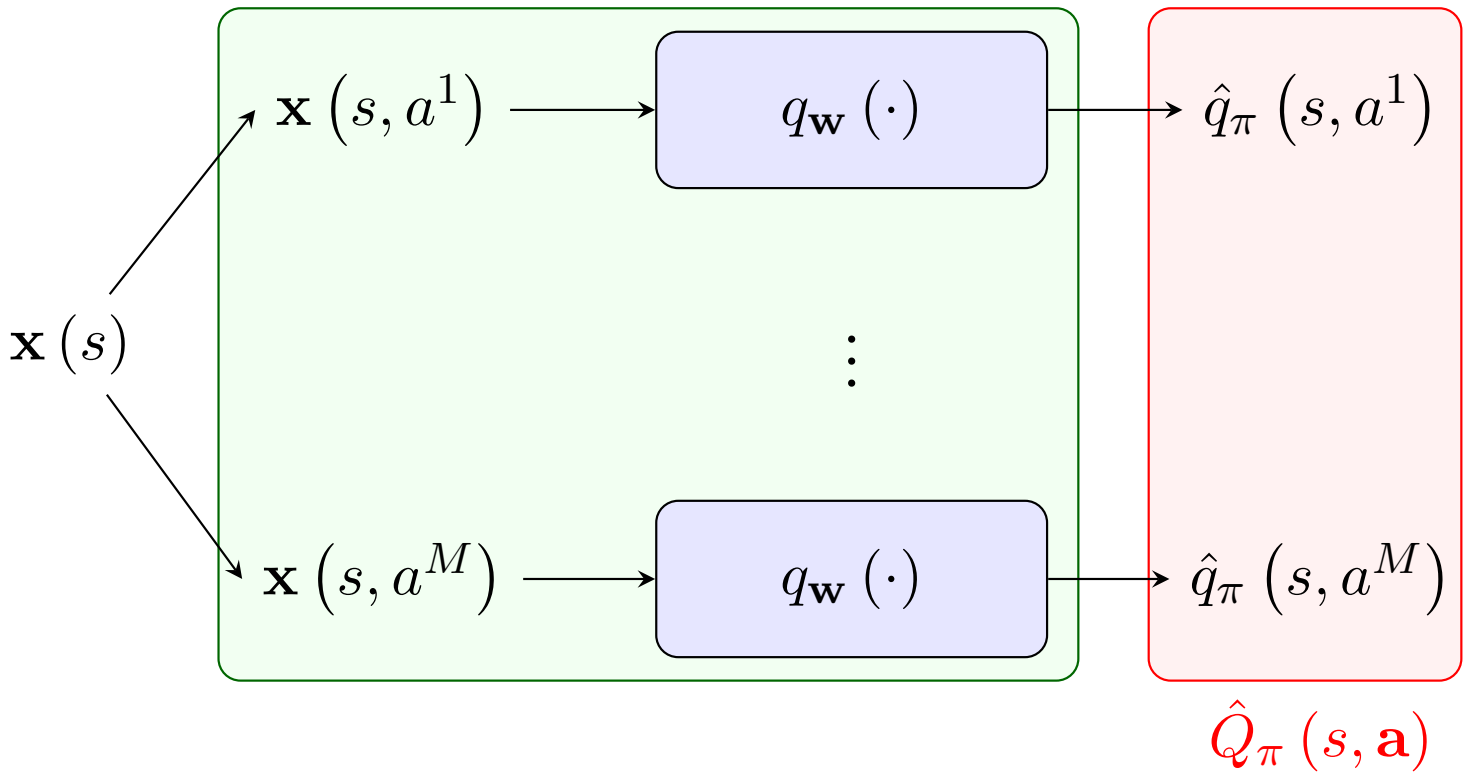
This is a very practical setting that show up in robotics, autonomous driving, etc

---

+ *What about it? Why should be a challenge to use DQL with continuous action space?*
– Well! How could we maximize action-value in this case?!

Let's take a look to see the challenge clearly

DQN: *Recalling the Output*

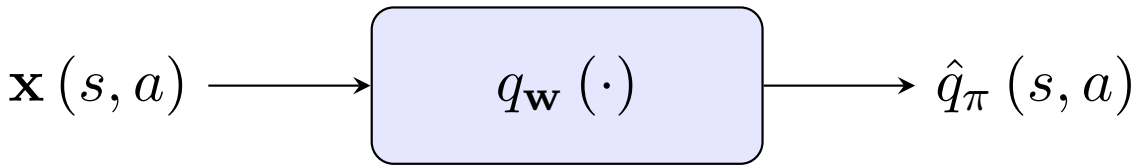


With continuous actions, we cannot enumerate the output!

- \+ *Why don't we use action-value approximator of form I?!*
- – Well! Let's do this

Recall: *Action-Value Approximator – Form I*

This is what we called Form I



Let's now see how the DQL algorithm can be applied

↳ *Let's consider the vanilla DQL*

1: *Update policy to* $\pi \leftarrow \epsilon$-`Greedy`$(Q_{\mathbf{w}}(S_t, \mathbf{a}))$
2: *Draw action* A_t *from* $\pi(\cdot|S_t)$ *and observe* $S_t, A_t \xrightarrow{R_{t+1}} S_{t+1}$
3: $\Delta \leftarrow R_{t+1} + \gamma \max_m Q_{\mathbf{w}}(S_{t+1}, a^m) - Q_{\mathbf{w}}(S_t, A_t)$
4: *Update* $\mathbf{w} \leftarrow \mathbf{w} + \alpha \Delta \nabla Q_{\mathbf{w}}(S_t, A_t)$

We obviously can replace $Q_{\mathbf{w}}(\cdot)$ *with* $q_{\mathbf{w}}(\cdot)$

DQL with *Action-Value Approximator – Form I*

1: *Update policy to* $\pi \leftarrow \epsilon\text{-}\texttt{Greedy}(Q_{\mathbf{w}}(S_t, \mathbf{a}))$
2: *Draw action* A_t *from* $\pi(\cdot|S_t)$ *and observe* $S_t, A_t \xrightarrow{R_{t+1}} S_{t+1}$
3: $\Delta \leftarrow R_{t+1} + \gamma \max_m Q_{\mathbf{w}}(S_{t+1}, a^m) - Q_{\mathbf{w}}(S_t, A_t)$
4: *Update* $\mathbf{w} \leftarrow \mathbf{w} + \alpha \Delta \nabla Q_{\mathbf{w}}(S_t, A_t)$

We can replace these updates with

1: *Update policy to* $\pi \leftarrow \epsilon\text{-}\texttt{Greedy}(q_{\mathbf{w}}(S_t, a))$
2: *Draw action* A_t *from* $\pi(\cdot|S_t)$ *and observe* $S_t, A_t \xrightarrow{R_{t+1}} S_{t+1}$
3: $\Delta \leftarrow R_{t+1} + \gamma \max_a q_{\mathbf{w}}(S_{t+1}, a) - q_{\mathbf{w}}(S_t, A_t)$
4: *Update* $\mathbf{w} \leftarrow \mathbf{w} + \alpha \Delta \nabla q_{\mathbf{w}}(S_t, A_t)$

Well! We need to optimize over a continuous variable!

+ *Where exactly we need it?*
– In lines 1 and 3: *once for* ϵ*-greedy update and once for off-policy control*

DQL with *Action-Value Approximator – Form I*

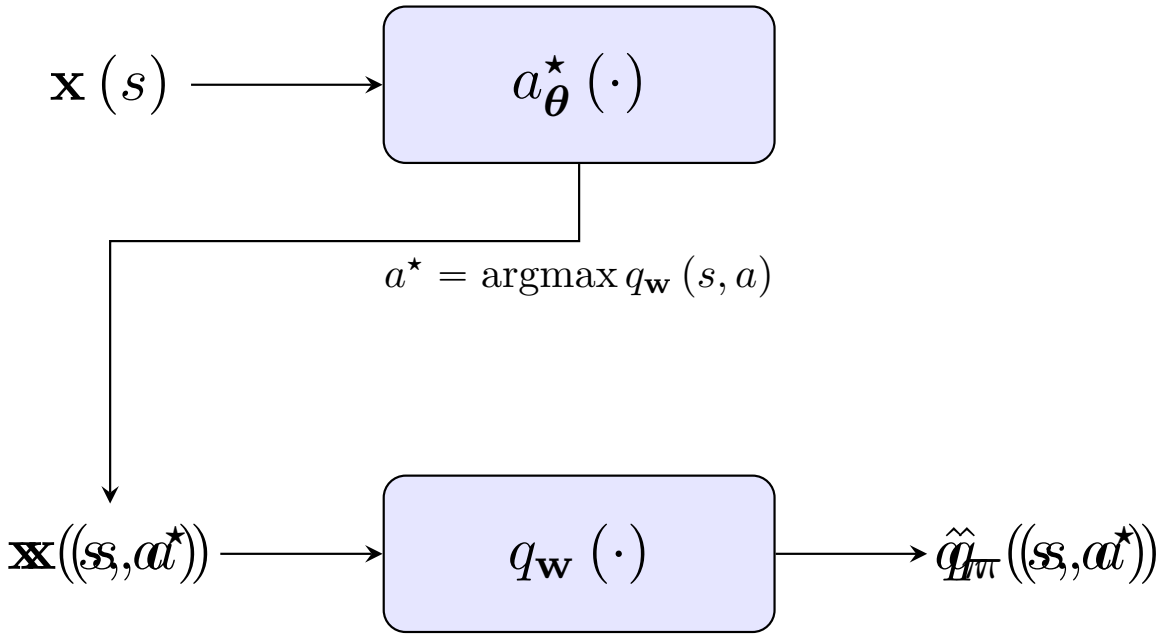
This is an essential challenge: *we need to optimize over a continuous variable*

- *We may grid the action space*
  - ↳ *It's computationally expensive: we are back at square one!*
- *We may apply gradient descent*
  - ↳ *It makes a two-tier loop: it's again computationally expensive!*

+ *It sounds like impossible!*
– *Only impossible is impossible*

In practice, we solve the target optimization via a DNN!

Learning Optimal Action



We could then update in DQL algorithm as

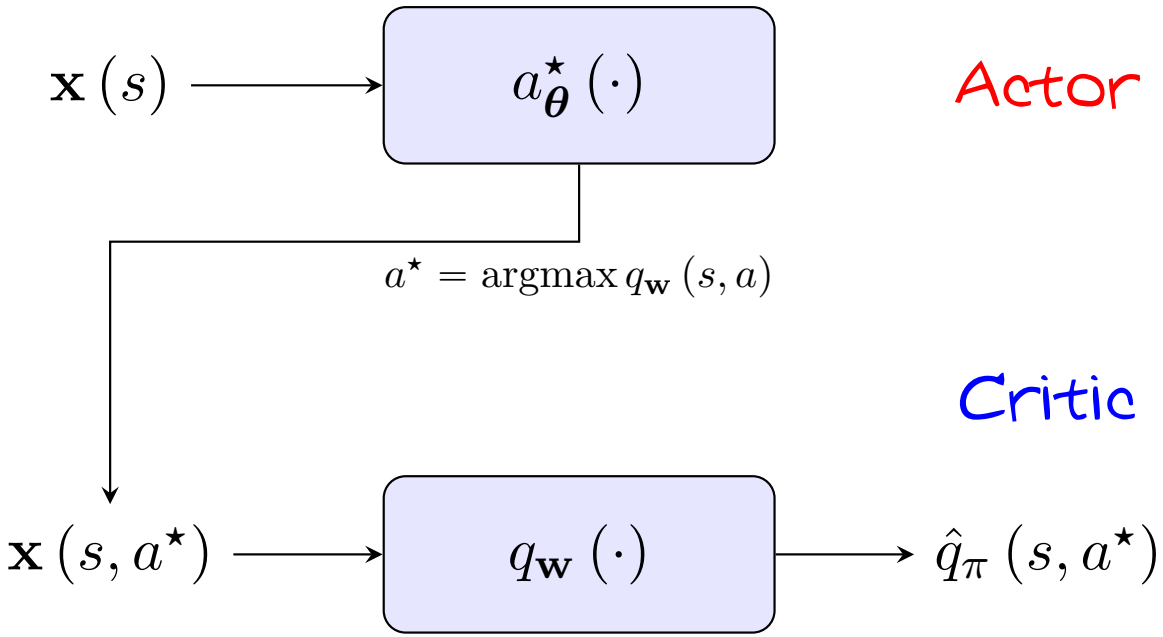
$$\Delta \leftarrow R_{t+1} + \gamma q_{\mathbf{w}}\left(S_{t+1}, a^{\star}\right) - q_{\mathbf{w}}\left(S_t, A_t\right)$$

and for ϵ-greedy improvement, we could

act $a^{\star}$ with probability $1 - \epsilon$ and random with probability ϵ

Learning Optimal Action

+ *Say we trained the network after some time; then, what do we do?*
- We act $a^\star$ for each state s



This is a particular example if actor-critic algorithm!

↳ *We will discuss these algorithms*

Solution to Continuous Action: *Policy Networks*

Our actor learns an optimal greedy policy

$$\mathbf{x}(s) \longrightarrow \boxed{a^{\star}_{\boldsymbol{\theta}}(\cdot)} \longrightarrow a^{\star}(s) \longrightarrow \begin{cases} 1 & a = a^{\star}(s) \\ 0 & a \neq a^{\star}(s) \end{cases}$$

This is a simplified form of the so-called policy network

$$\mathbf{x}(s, a) \longrightarrow \boxed{\pi_{\boldsymbol{\theta}}(\cdot)} \longrightarrow \hat{\pi}^{\star}(a|s)$$

Policy Network

Policy network is an approximation model that maps state-action features to the optimal policy

Solution to Continuous Action: *Policy Networks*

+ *What is the point in doing DQL anymore when we have a ? We already learn the optimal policy that we are looking for!*
– Great! This is what we do in *policy gradient algorithms*

Policy networks are used in two sets of deep RL approaches

- *Policy gradient approaches*
  - ↳ *We do not use a value network and directly approximate optimal policy*
  - ↳ *This is what we study next*
- *Actor-critic approaches*
  - ↳ *We keep the value network to examine the approximated optimal policy*
  - ↳ *This is the most practically-robust approach we can use*